Region Ownership: Unifying the Priority-Ceiling Protocol and DMA Handshakes in a Bare-Metal Language

Daniel Tralamazza*
Independent
tralamazza@gmail.com

Working draft — June 25, 2026

Working draft, not peer-reviewed. The evaluation is preliminary and being strengthened; the implementation is alpha and unproven. Comments welcome.

Abstract

A compiler for a microcontroller normally models a single bus master: the CPU. But memory correctness on a modern MCU is a property of the whole system — CPU cores, DMA engines, caches, and the bus matrix that connects them. The resulting bugs are silent: a buffer placed in memory the DMA engine cannot reach, a cache line the DMA never sees, a descriptor armed before the CPU has finished writing it. We describe BML, a language and compiler (beemel) for ARM Cortex-M firmware, built around two ideas. First, every bus master is a first-class *agent* with its own reach, cache view, and permissions, and CPU/DMA memory sharing is checked at compile time. Second — our central claim — the CPU priority-ceiling protocol (mutual exclusion between interrupt and thread contexts) and the asynchronous DMA release/reclaim handshake are *one concept, region ownership*, with the synchronization *mechanism* derived from the set of sharers — the programmer declares the sharing and the ownership window, not which exclusion protocol to emit. The model was built failure-first: every check traces to a specific hardware bug we provoked on a development board, and the result was validated back on silicon across three DMA architectures (ST crossbar, RP2350 per-channel, nRF51 fixed-block). We position the work against the priority-ceiling literature (which covers the CPU half only) and the formal DMA-isolation and OS memory-model literature (which treats DMA but never unifies it with CPU-side mutual exclusion). These are compile-time analyses plus a test suite, not formal proofs; we are explicit about the resulting trust boundary.

*beemel was developed iteratively with AI assistance; the design emerged from hardware bring-up rather than an up-front specification. The author is responsible for all claims; the complete development history and test suite are public.

1 Introduction

The mental model behind most embedded toolchains is that a program runs on one processor, and memory is a flat array that processor reads and writes. A compiler optimizes against that model and a linker places symbols into it. The model is wrong for any MCU with a DMA engine. A network controller, USB controller, or timer-driven peripheral DMA is a second bus master: it issues its own loads and stores, on its own clock, with its own view of memory — a different reachable address range over the bus matrix, and, on cached parts, a view that does not snoop the CPU's data cache.

When the CPU and a DMA engine disagree about a region of memory, the failure is typically silent. The program compiles and links. The board boots. And then it never transmits a packet, or it transmits stale data, or it locks up intermittently under load. The toolchain emitted no diagnostic because, in its model, there was only ever one actor touching memory.

This paper describes BML, a language for Cortex-M firmware whose compiler, beemel, models the second actor. Our contributions are:

- **The agent abstraction** (§3): a uniform treatment of CPU cores, DMA engines, debug probes, and external firmware as *agents* — bus masters with a declared reach, cache view, and permissions — with an admission schema that survived cross-vendor falsification (no seventh question emerged across three DMA architectures).
- **Region ownership** (§4): the central contribution. We show that the priority-ceiling protocol (CPU/CPU mutual exclusion) and the DMA release/reclaim handshake (CPU/DMA mutual exclusion) impose the *same ownership obligation*, and that a compiler can *derive* which exclusion mechanism to emit from the set of agents that share a region — so the two compose automatically when an agent and a CPU context share one buffer. To our knowledge no prior source language derives the two from a single sharer-set abstraction (§8).

- **Derived whole-system checks** (§5): bus-matrix reachability, cache-coherence enforcement by generated MPU regions, transfer extent, and volatility-by-provenance, each derived from placement and use rather than annotated.
- **An empirical, failure-driven evaluation** (§7): every check is traced to a concrete bug provoked on hardware, and the model is validated on three boards. We report the abstract-interpretation encodings that were required to discharge the DMA obligations in a non-relational domain.

We are deliberate about what this is not. BML has no mechanized semantics and no soundness theorem; its guarantees are compile-time analyses backed by an end-to-end test suite and optional static verification. §9 states the trust boundary precisely. We argue the contribution is a *model and its validation*, in the tradition of language-design experience reports. Figure 1 shows a complete BML fragment up front; the rest of the paper explains what each line checks.

2 Motivation: a CPU-only compiler is blind

The model in this paper was not designed up front. It was forced out by a sequence of failures on a NUCLEO-H723ZG (Cortex-M7) while bringing up an Ethernet driver, and refined against two further boards. The founding failures, all observed:

Unreachable placement. Moving the transmit descriptor ring into DTCM (tightly-coupled memory) compiled, linked, and produced a board that never transmitted. The Ethernet DMA cannot address DTCM over the bus matrix; nothing in the toolchain knew the bus topology, so nothing objected.

Unchecked cache discipline. Synchronizing the descriptor publish with a ‘dmb‘ barrier alone was sound only because the D-cache happened to be disabled. Enabling the cache would have broken receive silently — the DMA writes a descriptor the CPU never sees because the CPU reads a cached copy.

A posted-write completion hazard. Arming the DMA is a posted (buffered) write on a Device-memory register. One left in flight while the bus remained busy was an intermittent source of imprecise bus faults — present or absent depending on instruction scheduling.

An optimizer-hoisted ownership poll. A plain load of a descriptor's ownership bit, in a spin loop waiting for the DMA to finish, was hoisted by the optimizer out of the loop into an infinite branch-to-self. The CPU could not see that a concurrent writer (the DMA) existed.

kind	example	accesses answered for by
`cpu`	Cortex-M core	the compiler (every IR access)
`dma`	Ethernet DMA, EasyDMA	owning module, at handoffs
`debug`	SWD probe	host side; concurrency-inert
`external`	other-vendor firmware	nobody; channels only

Table 1: Agent kinds. A `cpu` agent's accesses are visible in the compiler's IR and checked per access; a `dma` agent's accesses are invisible, so the compiler sees only the address flowing into a register.

Each failure is a disagreement between two bus masters: about reachability, about visibility, about completion ordering. A compiler that models only the CPU has no way to state any of them, let alone check them. The rest of the paper is the vocabulary we added to state them and the checks that follow.

3 The agent model

An agent is anything that touches memory on its own initiative. The admission test is three questions, all of which must hold: (1) does it initiate memory accesses itself, rather than only decode them? (2) does it act concurrently, on its own clock, once set up? (3) does it have its own view of memory — reach, caching, permissions? A timer that triggers a peripheral DMA is an agent; the peripheral's own register file is not. The RP2350 PIO executes instructions but has no memory reach, so it fails question (3) and is not an agent.

The asymmetry in Table 1 is the crux. For a `cpu` agent the compiler emits every load and store, so its rules are ordinary per-access checks. For a `dma` agent the accesses are opaque — the compiler sees only an address written into a controller register. A *handoff* is that register write, and it is where the compiler recovers, for an opaque agent, the visibility it gets for free on the CPU.

The agent contract is, so far, a closed schema: every key in an agent declaration answers one of six questions about a bus master — may it run, where can it touch, which buffer, how much, when is it done, what code does it run. We arrived at the six by reviewing and instantiating three structurally different DMA architectures: ST's central crossbar with software-selected master ports (H723 Ethernet and MDMA), Raspberry Pi's per-channel controller (RP2350), and Nordic's per-peripheral fixed-block engines (nRF51, where every capable peripheral is its own bus master). No seventh question emerged across the three; the schema is closed only modulo the acknowledged future refinements of §9 (e.g. scatter-gather descriptor chains), not proven minimal.

```
import stm32h723.mdma;

owns MDMA;                                  // exclusive control of the engine

var SRC: [u8; 64] in dma_shared;            // MDMA reads this region
var DST: [u8; 64] @shared in tcm_scratch;   // MDMA writes; thread + ISR also read
var RESULT: u32;                            // CPU-visible result

fn copy() {
    var i: u32 = 0;                         // fill before release: index-WRITE is legal
    while i < 64 { SRC[i] = i as u8; i = i + 1; }

    MDMA.MDMA_C0SAR = &SRC as u32;          // release: hand the engine the source ...
    MDMA.MDMA_C0DAR = &DST as u32;          // ... and the destination
    MDMA.MDMA_C0CR.EN = true;               // arm

    claim DST {                             // CPU window: masks the ISR co-sharer
        if MDMA.MDMA_C0ISR.CTCIF0 {         // observe the engine's completion flag
            const v = reclaim(DST);         // only now is a bounds-checked view sound
            RESULT = v[0] as u32;           // (calls are forbidden inside a claim)
        }
    }
}
```

Figure 1: A complete BML fragment, condensed from the H723 MDMA example. `DST` is shared by the thread, an ISR, and the MDMA engine, so its carrier is `Shared(AgentShared([u8;64]))` and every access outside the window is rejected: dropping the `claim` makes `reclaim` an error (the ISR could race), dropping the flag test is a missing-handshake error, and an index-read of `DST` elsewhere is rejected because the engine may still own it. The `claim`/`reclaim` nesting — one CPU window, one agent handshake — is the unification of §4, emitted from the fact that a CPU context and a DMA agent share `DST`.

4 Region ownership

A *region* names a block of memory and the set of agents that share it. This section is the core claim: a CPU sharer and a DMA sharer impose the same ownership *obligation* — exclusive ownership with observed transfer — and the compiler derives the (distinct) exclusion mechanism from the sharer set.

4.1 Two masters, one ownership

Consider a buffer shared between the CPU and one other agent. Correct access requires that at any moment at most one of them treats the buffer as live, and that ownership transfers are observed by both sides. This is mutual exclusion, and it has two textbook instantiations that are normally studied in separate literatures:

- If the other sharer is **another CPU context** (an ISR sharing a buffer with a thread, or a higher-priority ISR), exclusion is achieved by raising the processor's priority above every context that uses the buffer — the *priority-ceiling protocol* [1]. The owner holds the buffer for the duration of a masked critical section.
- If the other sharer is a **DMA engine**, exclusion is achieved by an *asynchronous handshake*: the CPU writes the buffer's address into a controller register (*release*), the engine works on it, and the CPU must observe a completion signal before touching the buffer again (*reclaim*).

We claim these share one *obligation* — region ownership: at most one live owner, transfers observed by both. The two *mechanisms* remain genuinely different — ceiling masking is preventive and synchronous (the contending context cannot run), the DMA handshake is cooperative and asynchronous (release now, observe completion later) — and which one the compiler emits is a function of *who* the co-sharer is. The contribution is not that the mechanisms are identical, but that one obligation, derived from the sharer set, subsumes both and makes their composition automatic.

4.2 The CPU sharer: the claim window

When a region is shared between CPU contexts, BML attaches a *ceiling* to the shared static, computed from the set of contexts that access it. A `claim X { ... }` block opens an ownership window: on ARMv7-M it raises `BASEPRI` to the static's ceiling, so unrelated higher-priority interrupts keep running while every *contending* context is masked; on ARMv6-M, which lacks `BASEPRI`, it falls back to a global `cpsid`/`cpsie` pair. Inside the window the static is at its inner type — ordinary reads and writes are legal — and the per-access critical sections the compiler would otherwise insert are suppressed, because the window already provides exclusion.

4.3 The DMA sharer: release and reclaim

When a region is shared with a DMA agent, an array placed in it is given a move-typed carrier at name resolution. *Release* is the handoff register write. *Reclaim* is an explicit `reclaim(BUF)` that yields a bounds-checked view and is legal only once the channel's completion flag has been observed. The completion guard is checked by lexical containment: the `reclaim` must sit within the guard span of an observation of the channel's own `completes_by` flag (a conservative, non-flow-sensitive check) (Table 2). A plain view over move-typed memory, without the handshake, is rejected and the diagnostic points at `reclaim`.

4.4 Deriving the mechanism, and composing both

The programmer does not choose between `claim` and `reclaim`. The compiler reads the region's sharer set and emits the mechanism: a masked window for a CPU co-sharer, a release/reclaim handshake for a DMA co-sharer. When both are present the carriers nest, and unguarded access becomes unrepresentable in well-typed code: under a static that is both ceiling-shared and DMA-shared, *every* access — reads included — is rejected outside a `claim`, so the masked window is required by construction and the handshake composes inside it with no additional rule:

```
claim X {            // CPU co-sharers masked here
    if done() {      // DMA co-sharer's completion
     observed
        reclaim(X)   // only now is a view over X sound
    }
}
```

The same observation runs the other way at the `verify` layer (§6): a read of the claimed static *inside* the window is discharged as stable by IKOS (the mask excludes the only writer), while the identical read outside the window is not. Ownership is the single fact that both the type checker and the abstract interpreter consult.

5 Whole-system checks derived from the model

The ownership story rests on a layer of checks that connect placement to physics. They are organized so that the truth lives in exactly one place: chip *physics* in a target file, written once per part; project *policy* as regions; and *software binding* in the source. The governing principle is that the program declares only what use cannot reveal — the bus matrix and exclusivity claims — and the compiler derives the rest.

```
[mem.dma_pool]
base = 0x30007000
size = 4K
cacheable = false       # -> generated MPU region

[agent.mdma]
kind        = dma
reach       = dma_pool, dtcm   # checked vs. bus
bus         = axi: 0x08000000..0x08100000,
              ahbs: 0x00000000..0x00040000
enabled_by  = RCC.C1_AHB3ENR.MDMAEN

[agent.mdma.ch0]
completes_by = MDMA.MDMA_C0ISR.CTCIF0
extent       = MDMA.MDMA_C0BNDTR.BNDT
handoff      = MDMA.MDMA_C0SAR
               port_by MDMA.MDMA_C0TBR.SBUS ahbs

[region.dma_shared]
mem    = dma_pool
agents = mdma           # CPU always implicit
```

Table 2: A target-file fragment (Layer 1, physics). No key takes a raw address: every register is an SVD path, so a stray literal fails at resolution rather than silently configuring hardware.

Bus-matrix reachability. A region's memory must lie within every listed agent's reach, and `reach` is itself cross-checked against the transcribed `bus` windows. The DTCM placement bug from §2 fails at target load, before any source is compiled.

Cache coherence as generated protection. A cacheable block shared by a cached CPU and a non-snooping DMA agent is rejected. Declaring it `cacheable = false` does not merely annotate: it *generates* a non-cacheable MPU region in the reset handler (PMSAv7 on Cortex-M4/M7, PMSAv8 on M33). Alignment of a static in such a region is floored to the CPU's cache-line size — the alignment requirement was always a property of the cache, never of the DMA engine.

Transfer extent. The `extent` field names the controller's transfer-count register; at the `verify` layer, arming the channel emits an obligation that `count` × `unit` does not exceed the delivered buffer's capacity.

Volatility by provenance. BML has no `volatile` keyword. Volatility is a property of where data lives: a pointer derived from the address of a DMA-shared static points at memory with a concurrent writer the optimizer cannot see, so every load and store through such a pointer is lowered as volatile. The taint is tracked syntactically from the address-of site to a fixpoint, and an escape check (the pointer may not leave the deriving function) prevents it from being silently lost; an address laundered through integer arithmetic escapes the syntactic taint and is a recorded blind spot (§9). This directly

addresses the optimizer-hoisting failure of §2, and is a concrete response to the long-standing observation that `volatile` is fragile and miscompiled [3]. Crucially, a view obtained *inside* a justified ownership window stays non-volatile: the agent is excluded there, so the proof restores the optimization that the unguarded boundary forbids.

6 Optional verification

6.1 The pipeline

`bml verify` lowers the program to LLVM IR and runs IKOS [13], an abstract-interpretation analyzer, for buffer overflows, null dereferences, division by zero, integer overflow, and user assertions. The regions model adds obligations on top: handoff provenance and reach, in-memory descriptor pointer placement, and transfer extents.

6.2 Encodings the domain forced

Discharging the DMA obligations in IKOS's default non-relational interval-with-congruence domain required specific encodings, which we report because they are reusable lessons for anyone driving abstract interpretation at this layer:

- The provenance `assume` must sit at the address-of site. Two integer casts of the same global do not unify across function boundaries, so an assume placed at the use site proves nothing.
- Capacity is encoded as a *constant* compared against a count interval. A base/limit *pair* of shadow variables is unprovable in a non-relational domain; a constant-versus-interval comparison is trivial in it.
- Alignment is carried by exact addresses, not congruences. The domain does no backward congruence narrowing, and because the compiler owns the linker script, the exact address is available and sufficient.
- Claim-aware havoc. Reads of a `@shared` static are normally invalidated before each load (a higher-priority writer may have run); reads of the *claimed* static inside its window are not, because the mask provides exactly that stability. This is the ownership fact of §4 seen by the analyzer.

7 Evaluation

We offer three kinds of evidence: that BML's checks reject hazards *independently documented* for chips (§7.1) and *third-party drivers* (§7.2) it was not co-designed with; a controlled counterfactual reproduced on hardware (§7.3); and end-to-end bring-ups on three boards (§7.4). The first two directly address the central threat to an experience paper — that the checks were exercised only on the programs they were built against.

7.1 Documented hazards caught at compile time

Cortex-M firmware has a small family of notorious DMA bugs that vendors document in application notes and that recur for years in forums and issue trackers. We took four, spanning three vendors, and reconstructed each minimally in BML — the documented memory-sharing structure, not the original C, which BML cannot ingest. In every case the compiler rejects the program; Table 3 summarizes, and the moved-to-reachable-memory counterpart of each builds clean.

Two observations. First, a single check — the Layer-2 reach test — catches the same bug class (a buffer the engine cannot address) across three structurally different bus arrangements (ST's DTCM, ST's CCM, Nordic's flash exclusion): the agent/reach model's generality made concrete rather than asserted. Second, the "caught today by" column is the point. These hazards are presently caught at *runtime* (a hard fault), *not at all* (silent corruption), or by *manual* configuration one can omit; BML moves them to compile time, and for the cache hazard it *generates* the non-cacheable MPU region that ST's examples write by hand.

Honest scope. These are reconstructions, not analyses of the upstream drivers: we re-expressed each documented hazard's placement and sharing in BML and confirmed the diagnostic by running the compiler. The reach test is only as good as the transcribed bus matrix, which lives in the shipped chip target (§5); a mis-transcribed reach window would mask the bug. The model also does not capture every subtlety — the Cortex-M7 write-through-cache erratum and speculative-read concerns of AN4839 [20] are outside it. The claim is narrower and concrete: the common, high-frequency form of each hazard becomes a compile error.

7.2 Third-party drivers: the same constraint, by hand

The catalog cites vendor application notes; the same hazards recur in *third-party* drivers BML never saw, where named maintainers hit them and added a guard — weaker and later than a compile error. Two cases, each reconstructed in BML and anchored to the project's own issue:

`stm32-rs/stm32-eth` (Rust), issue #16 [23]. The driver's maintainers documented that descriptors and buffers in a D-cache-enabled region "would need cleaning ... and invalidating," and proposed to *require* that they not be placed in such a region. Reconstructing a cacheable TX descriptor ring handed to the ETH DMA, BML rejects it at target load (*"region `eth_ring` ... cache views diverge"*) and, with `cacheable = false`, generates the non-cacheable MPU region that otherwise has

Documented hazard (chip)	Attested by	Caught today by	BML, at compile time
DMA buffer in DTCM (STM32H7)	AN4839 [20]; ST community	nothing: board silently never transmits	rejected at target load: *"region in mem* `dtcm`*, which agent* `eth` *cannot reach"*
DMA buffer in CCM RAM (STM32F4)	ST community; NuttX `CCMEXCLUDE`	runtime AHB / hard fault	rejected: *"mem* `ccm` *... agent* `dma2` *cannot reach"*
EasyDMA buffer in flash (nRF52)	Nordic spec [21]; `nrf-hal` #37	hard fault or silent; embassy's run-time `BufferNotInRAM`	rejected: *"mem* `flash` *... agent* `easydma` *cannot reach"*
Cacheable DMA buffer, D-cache on (STM32H7, SAME70)	AN4839 [20]; Zephyr #36471 [22]	silent corruption ("works with cache off")	rejected: *"cache views diverge"*; the fix `cacheable=false` auto-generates the non-cacheable MPU region

Table 3: Independently-documented, multi-vendor DMA hazards, each reconstructed in BML and rejected by the compiler. One Layer-2 reach check (rows 1–3) catches the same bug class across three bus architectures; the cache check (row 4) both rejects the program and generates the vendor-prescribed fix. Reconstructions and verbatim compiler output are in the artifact.

to be arranged by hand (a dedicated linker section plus a compile-time placement assertion — the standard defensive pattern).

`nrf-rs/nrf-hal` and embassy (Rust), issue #37 [24]. "EasyDMA doesn't read from Flash": a flash-resident `&'static` buffer handed to a UARTE transmit failed *silently*. The fix was a *run-time* check returning `BufferNotInRAM` (embassy additionally offers a variant that copies the data into a RAM scratch buffer). Reconstructing the same delivery, BML rejects it at target load (*"region* `tx_buf` *in mem* `flash` *... agent* `easydma` *cannot reach"*) — the same physical fault, moved from run time to compile time, with no copy.

The pattern is the point: independent projects converged on BML's constraint and enforced it the only ways a library can — a runtime check, a defensive copy, or a documented assertion — where BML makes it a property the compiler checks once. The honest scope of §7.1 carries over: these reconstruct the documented memory-sharing structure, not the upstream source, which BML cannot ingest.

7.3 Falsified both ways

The volatility-by-provenance check of §5 is a controlled counterfactual on hardware. Under a plain (non-volatile) load, the Ethernet controller's ownership-bit poll froze at transmit frame 5 — the optimizer had hoisted the load out of the spin loop into a branch-to-self. Under the agent-derived volatile lowering, the identical program runs indefinitely. The bug and its fix were each reproduced on the board, not only in the IR.

7.4 Hardware bring-ups

Three boards span the three DMA architectures of §3. These are existence proofs that the model is self-consistent and runs end to end; they do not, on their own, isolate the unification claim (§9).

NUCLEO-H723ZG (Cortex-M7). Ethernet transmit and receive with typed descriptors, D-cache *enabled* under the generated MPU region. An MDMA copy into DTCM routes through the software-selected bus port. The full claim/reclaim composition holds an exact cross-context invariant (LOG_SUM $= 4 \times$ TICKS $- 10$) over a long run. Descriptor extents are proven at the `verify` layer and confirmed at run time by the DMA's write-back of the transferred length.

Pico 2 W (dual Cortex-M33). First-flash boot via the Arm `IMAGE_DEF` block; the generated PMSAv8 MPU registers read back exactly as emitted; the core1 launch handshake; and a cross-core `claim` backed by a hardware spinlock sustaining tens of millions of contended ownership windows with no observed in-window invariant violation.

micro:bit v1 (nRF51, Cortex-M0). An AES-ECB known-answer test driven end to end through the full chain — handoff, fixed-size extent, guarded reclaim — passing on first flash, with the ARMv6-M word-composed interrupt-priority path exercised.

8 Related work

The components of our model exist in prior work; the unification does not.

CPU mutual exclusion. The priority-ceiling and Stack Resource Policy [1] are the CPU-side half of our ownership story, and they are well established: Ada's Ravenscar profile [19] implements ceiling locking, and RTIC [2] brings it to embedded Rust with `BASEPRI`-based critical sections computed from the set of sharing tasks. None of this work extends the protocol to a non-CPU sharer or an asynchronous completion handshake.

DMA as a principal with a memory view. The closest prior art to our agent abstraction is the ETH Zurich decoding-net / least-privilege memory model [4], which treats DMA engines and cores as first-class subjects in a network of address spaces and even precomputes fixed-topology translations at compile time. It is, however, an operating-system memory-management model enforced by a kernel reference monitor through runtime capabilities (implemented in Barrelfish, with a sketch for Linux); its reachability machinery *enables* an access by finding a translation path, assuming reconfigurable MMU/IOMMU hardware. BML targets bare metal with no such hardware: reach is a fixed property of the bus matrix, and an unreachable placement is a compile error with no path to route around it. The decoding-net work models DMA as a principal but never connects it to CPU-side mutual exclusion.

DMA buffer handoff. Embedded Rust encodes handoff as affine move semantics: a `Transfer` type consumes the buffer by value and returns it only after a busy-wait on completion [5]. This is a library idiom rather than a language-level model; it does not check transfer length against buffer capacity, and reachability is assumed. Crucially, RTIC and embassy [18] already provide both ceiling-based locking and typestate DMA ownership in one framework — but as two distinct, programmer-chosen mechanisms: neither derives the choice from the sharer set, and neither forces the DMA handshake to nest inside the ceiling window when a buffer is shared both ways. That single derivation, and the composition it makes automatic, is our delta over them. Per-driver functional proofs, such as the Pancake verification of an i.MX Ethernet driver [6], verify one descriptor ring in program logic, single-threaded, with no ceiling protocol.

Formal DMA isolation. Haglund and Guanciale [7] give a HOL4 framework that proves the conditions under which a DMA controller confines its accesses to permitted memory regions — a security-policy property, discharged by interactive theorem proving for a driver or runtime monitor. The framework models neither caches nor CPU-side concurrency, and it verifies a runtime configuration rather than checking a source program.

Ownership types and verified DMA drivers. Refined ownership types for C — RefinedC [14] and CN [15] — formalize exactly the ownership/capacity discipline our move-typed carriers and extent obligations approximate, but as a general separation-logic framework, not a bus-master model with reach, caches, or a ceiling protocol. Cogent [16] brings linear types to verified systems code (file systems, not DMA paths). In the seL4 line, a verified ZynqMP DMA driver in concurrent separation logic [17] treats device and driver as concurrent agents over shared state — the closest framing to ours — but verifies one driver per proof and does not unify the device handshake with CPU-context exclusion.

Static analysis and compiler-mediated DMA isolation. The closest prior statement of our founding observation — that one cannot verify a driver without modeling the asynchronous device — is Monniaux [8], who verified an industrial USB OHCI driver by abstract interpretation, composed asynchronously with a hand-written model of the DMA-capable controller, to prove absence of run-time errors of exactly the kind where "incorrect programming of the device" leads to "memory zones being erased." This is general-purpose abstract interpretation of a single driver-plus-device model, not a language in which bus masters are first-class agents: it does not derive reachability, model caches, or relate the device handshake to CPU-side mutual exclusion. PISTIS [9] isolates DMA on low-end MCUs without an IOMMU using a small trust anchor together with a compiler pass that inserts mediation around accesses to DMA-controller registers; enforcement is at run time by the monitor, not a compile-time property of the program text.

Safe-language operating systems. Singularity [10] made device interfaces type-safe but explicitly placed DMA outside its safe model, identifying it as the one inherently unsafe part of the driver/device interface. Ivory and Tower [11] give a memory-safe embedded DSL whose "regions" are allocation/effect regions in the Tofte–Talpin sense, not hardware-shared memory; Tock [12] isolates untrusted capsules and processes with the MPU at run time. None unifies a CPU ceiling protocol with a DMA handshake.

Summary. Across the priority-ceiling, OS memory-model, formal DMA-isolation, static-analysis, and safe-language-OS literatures, the components recur but the unification does not: we found no source language that joins CPU-side mutual exclusion and the DMA release/reclaim handshake into one ownership abstraction, derived from the sharer set and checked at compile time. That unification is the contribution.

9 Limitations and threats to validity

Checks, not proofs. BML's compile-time analyses, its end-to-end test suite (programs are executed under QEMU), and its optional IKOS verification reduce a class of bugs. They are not a soundness guarantee. The compiler has no mechanized semantics and has not been audited; it can have bugs of its own.

Recorded blind spots. We maintain an explicit register of what is trusted or lexically invisible. Among the entries: an address laundered through integer arithmetic defeats the syntactic volatile taint; system-exception priorities (SysTick, PendSV) are unmodeled; a completion flag cleared through a *separate* register is invisible to the straight-line staleness check; and a fact carried across a loop back-edge is outside the lexical windows — which today yields a concrete false positive: a correct continuous-copy DMA loop whose completion flag is cleared through a separate register (the STM32 IFCR idiom) is rejected. Each is documented where it was discovered rather than elided.

Scope. The implementation targets 32-bit ARM Cortex-M only. The cross-vendor falsification covers three DMA architectures, which is evidence for the schema's generality but not a proof of it; a controller with a structurally new "question" would extend the contract.

Evaluation. The evaluation is a set of hardware bring-ups, not a controlled bug-finding study against a corpus of existing firmware. The strongest threat to the central claim is that the model's value has been demonstrated on the programs it was co-designed with; a study applying the checks to independent third-party drivers is future work and would materially strengthen the case.

10 Conclusion

A microcontroller is a small distributed system: several bus masters, several views of memory, no shared clock. Treating it as a single processor with a flat memory is the source of a class of silent firmware bugs. BML treats every bus master as an agent and, more pointedly, observes that the CPU and the DMA engine want the *same* thing from a shared buffer — exclusive ownership, with observed transfers — and differ only in the mechanism, which the compiler can derive from who is sharing. Pushed against real silicon across three DMA architectures, the model suggests that whole-system memory correctness on an MCU is a checkable property and that one ownership obligation spans both masters. The implementation is alpha and the model unproven; §9 states the trust boundary.

Availability. beemel, the BML language specification, the example boards, and the verification toolchain are open source under Apache-2.0 at `https://github.com/tralamazza/beemel`.

References

[1] T. P. Baker. Stack-based scheduling of realtime processes. *Real-Time Systems*, 3(1):67–99, 1991.

[2] J. Aparicio, P. Lindgren, et al. RTIC: Real-Time Interrupt-driven Concurrency. `https://rtic.rs`.

[3] E. Eide and J. Regehr. Volatiles are miscompiled, and what to do about it. In *Proc. EMSOFT*, 2008.

[4] R. Achermann, N. Hossle, L. Humbel, D. Schwyn, D. Cock, and T. Roscoe. A least-privilege memory protection model for modern hardware. arXiv:1908.08707, 2019.

[5] The Embedonomicon: A no-std DMA API. Rust Embedded Working Group. `https://docs.rust-embedded.org/embedonomicon/dma.html`.

[6] Verifying device drivers with Pancake. arXiv:2501.08249, 2025.

[7] J. Haglund and R. Guanciale. Formally verified isolation of DMA. In *Proc. FMCAD*, 2022.

[8] D. Monniaux. Verification of device drivers and intelligent controllers: a case study. In *Proc. EMSOFT*, pages 30–36, 2007.

[9] M. Grisafi, M. Ammar, M. Roveri, and B. Crispo. PISTIS: Trusted computing for low-end embedded systems. In *Proc. USENIX Security Symposium*, 2022.

[10] G. Hunt and J. Larus. Singularity: Rethinking the software stack. *ACM SIGOPS Operating Systems Review*, 41(2), 2007.

[11] P. C. Hickey, L. Pike, T. Elliott, J. Bielman, and J. Launchbury. Building embedded systems with embedded DSLs. In *Proc. ICFP*, 2014.

[12] A. Levy, B. Campbell, B. Ghena, et al. Multiprogramming a 64 kB computer safely and efficiently. In *Proc. SOSP*, 2017.

[13] G. Brat, J. A. Navas, N. Shi, and A. Venet. IKOS: A framework for static analysis based on abstract interpretation. In *Proc. SEFM*, 2014.

[14] M. Sammler, R. Lepigre, R. Krebbers, K. Memarian, D. Dreyer, and D. Garg. RefinedC: automating the foundational verification of C code with refined ownership types. In *Proc. PLDI*, 2021.

[15] C. Pulte, D. C. Makwana, T. Sewell, K. Memarian, P. Sewell, and N. Krishnaswami. CN: Verifying systems C code with separation-logic refinement types. In *Proc. POPL*, 2023.

[16] S. Amani et al. Cogent: Verifying high-assurance file system implementations. In *Proc. ASPLOS*, 2016.

[17] BlueRock Security. Verified ZynqMP DMA driver in concurrent separation logic. seL4 Summit (presentation), 2025.

[18] The `embassy` project and `embedded-dma`. `https://embassy.dev`.

[19] A. Burns and A. Wellings. *Concurrent and Real-Time Programming in Ada* (the Ravenscar profile). Cambridge University Press, 2007.

[20] STMicroelectronics. AN4839: Level 1 cache on STM32F7 and STM32H7 series. Application note.

[21] Nordic Semiconductor. nRF52832 Product Specification — EasyDMA ("EasyDMA is not able to access the Flash").

[22] Zephyr Project. DMA and data cache coherency on Arm M7 devices. GitHub issue zephyrproject-rtos/zephyr #36471.

[23] stm32-rs/stm32-eth. Document possible problems when placing buffers on a cacheable region. GitHub issue stm32-rs/stm32-eth #16.

[24] nrf-rs/nrf-hal. EasyDMA doesn't read from Flash. GitHub issue nrf-rs/nrf-hal #37.